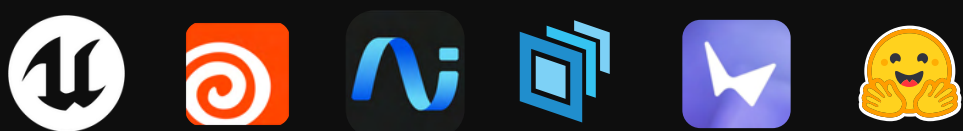


Sketch to Screen

AI Voxel Studio – Gaussian Splatting with USD Pipeline Integration



OVERVIEW



Render, Unreal Engine

ABSTRACT

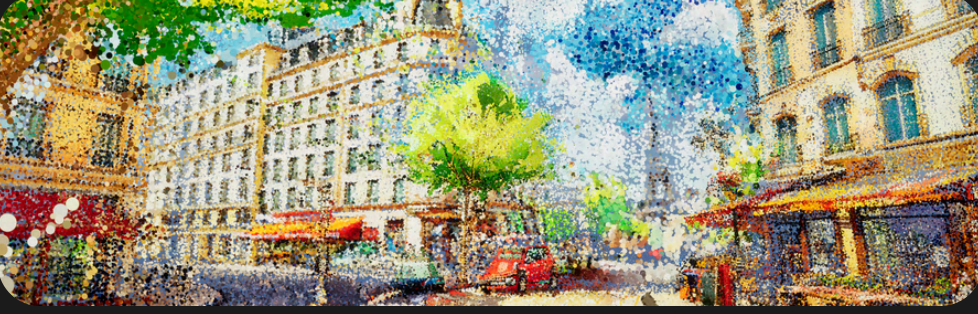
The work presents a production-ready pipeline converting Gaussian Splatting reconstructions into artistically controllable voxel environments from sketch to immersive world.

PROBLEM



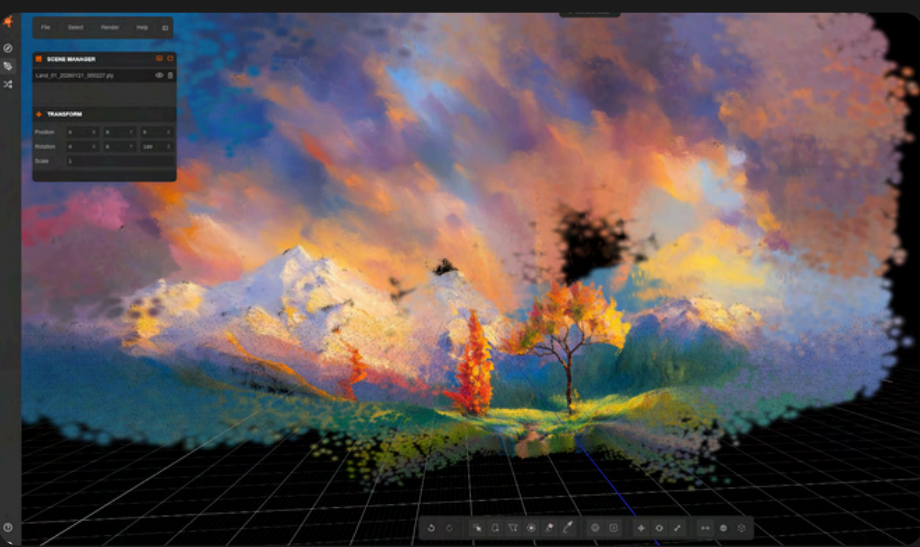
- Gaussian Splatting lacks artistic control
- No standardized 3DGS-to-UE pipeline
- Gaussian Splats difficult to edit and iterate
- USD typically used for export
- Limited real-time solutions for 3DGS

SOLUTION

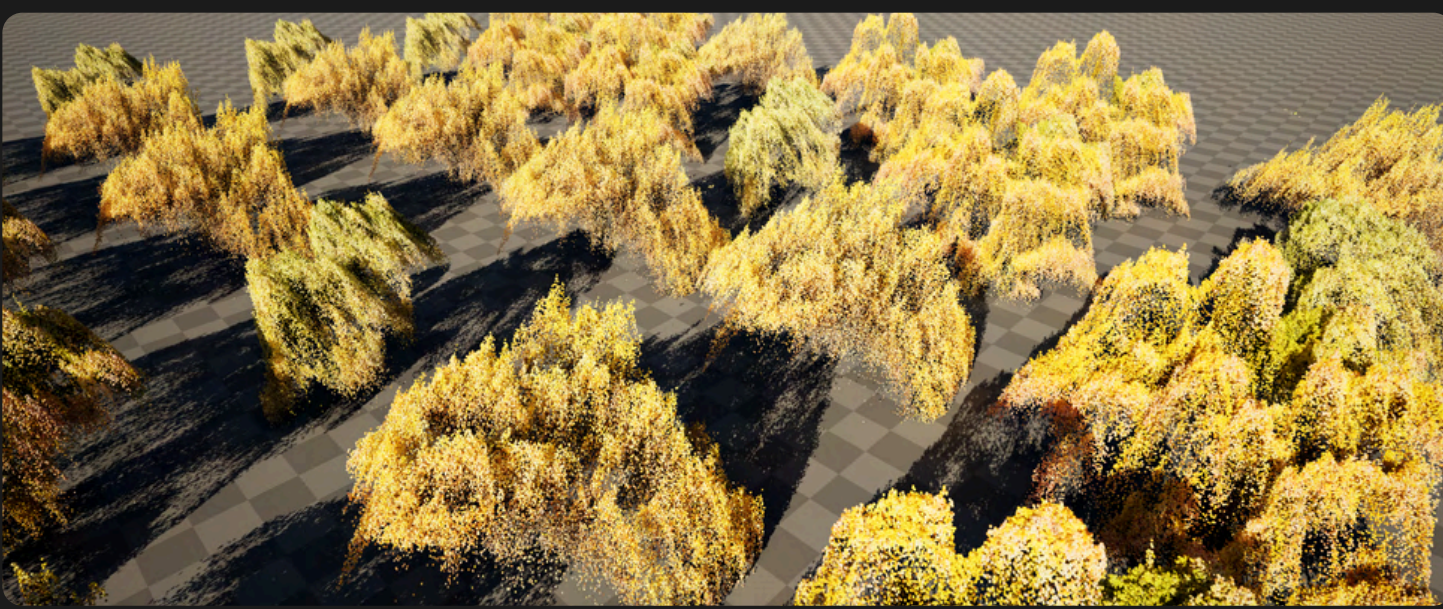


- Custom AI-driven image stylization (SDXL)
- Custom PLY-to-USD conversion tool
- GSOPs-optimized 3DGS workflows
- Instanced variation system
- Real-time integration in UE5 or anywhere

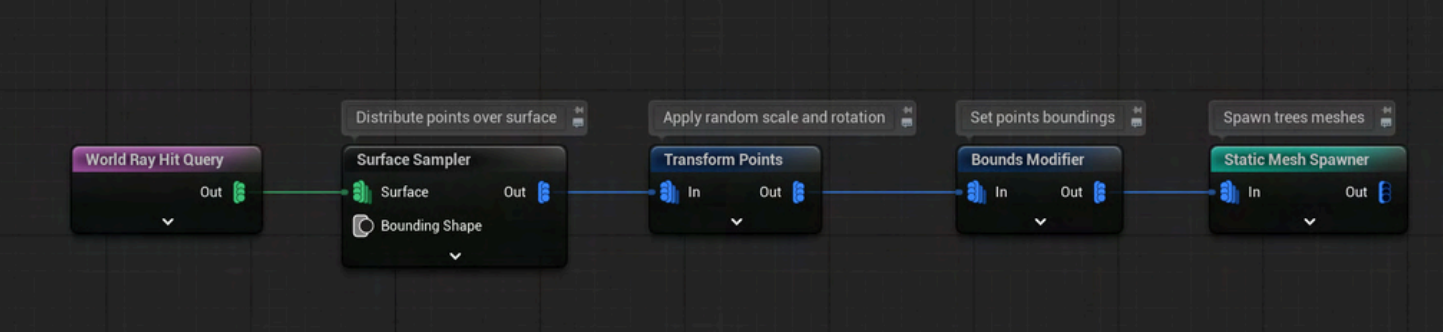
KEY POINTS



3DGS Clean up



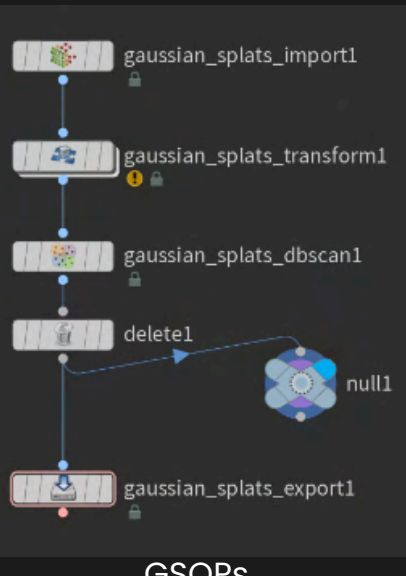
PCG Trees



PCG Graph



Collision Capabilities



GSOPs

- Potential to modulate (PCG)
- GSOPs - Optimize 3DGS
- Collision Capabilities
- USDC - 70% size reduction
- XR-Ready Performance (90+ FPS)

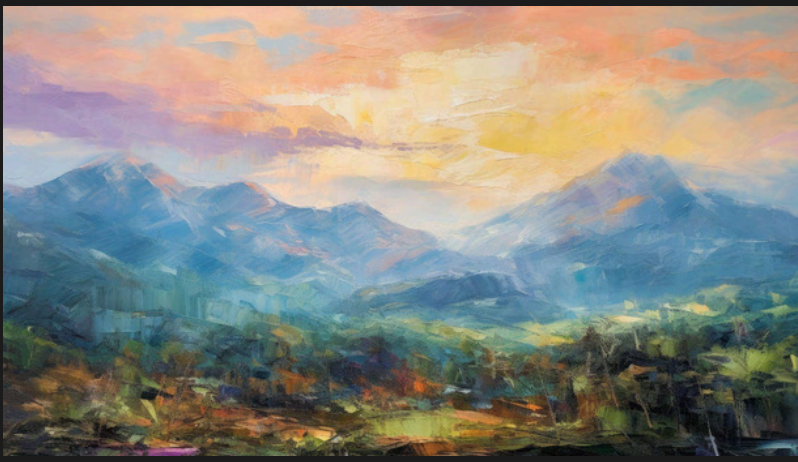
PIPELINE

01

AI STYLIZATION



Source image



Style Reference



Result

- Custom Tool, HP AI Studio
- SDXL + H94/IP-Adapter + ControlNet

02

3DGS GENERATION



Marble AI Lab

- Image to Gaussian Splat (PLY)
 - Marble AI Lab
 - Apple SharpML
- Gaussian Splats clean up
 - Super Splats
 - GSOPs in Houdini

03

USD CONVERSION

```
# Calculate bounds
min_bounds = positions.min(axis=0)
max_bounds = positions.max(axis=0)
grid_size = np.ceil((max_bounds - min_bounds) / resolution).astype(int) + 1

# Convert to grid coordinates
grid_coords = ((positions - min_bounds) / resolution).astype(int)

# Accumulate per voxel
voxel_data = Dict[Tuple[int, int, int], dict] = {}
for i in range(len(grid_coords)):
    key = tuple(grid_coords[i])
    if key not in voxel_data:
        voxel_data[key] = {
            'color_sum': np.zeros(3, dtype=np.float64),
            'count': 0
        }
    voxel_data[key]['color_sum'] += colors[i].astype(np.float64)
    voxel_data[key]['count'] += 1
```

- USD Instance (GPU optimized, true voxel look)
- Merged Geometry (single mesh)

Instance Options

Shape: ☒ Cube ☐ Sphere ☐ Cross

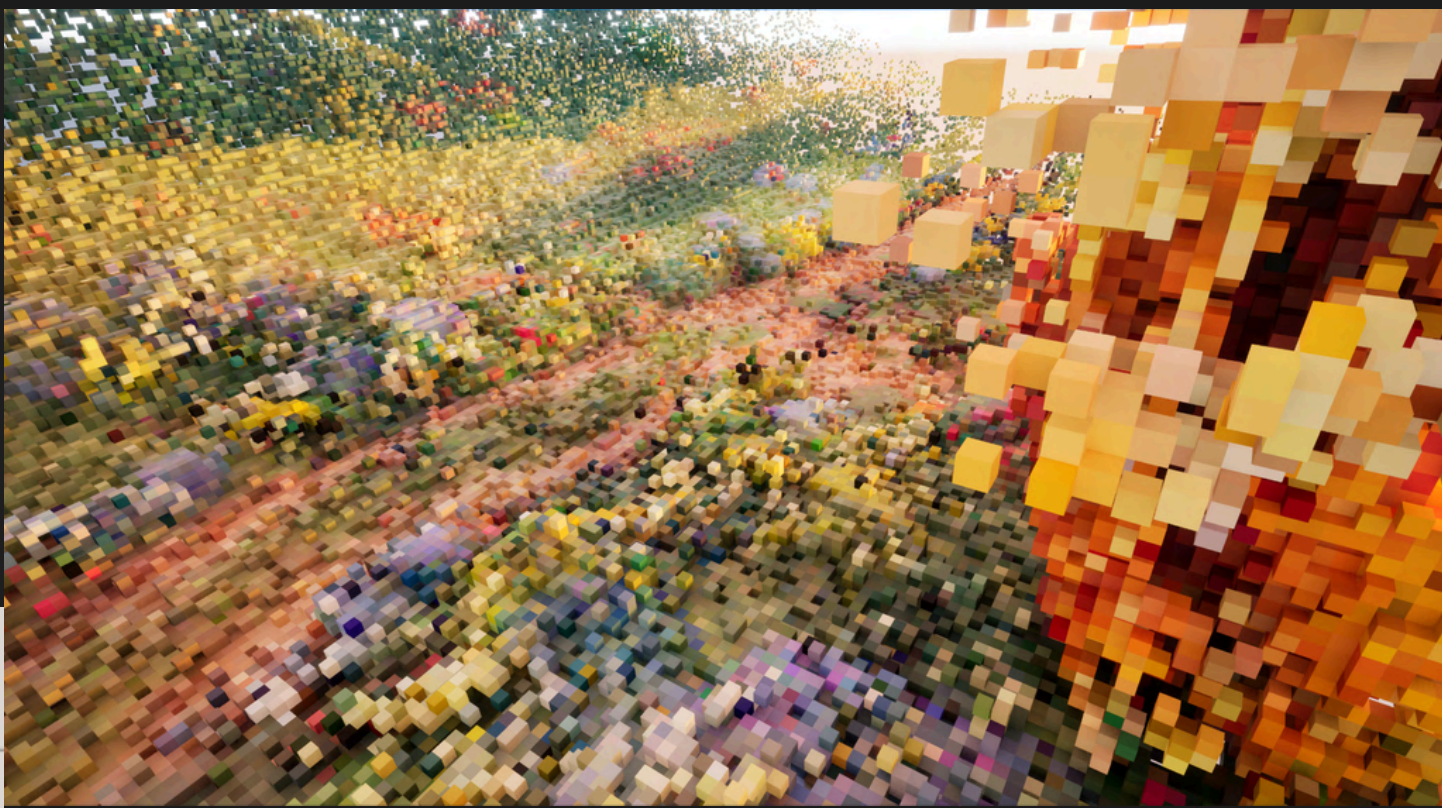
Color Quantize: ☒ 8 ☐ 16 ☐ 32 ☐ None

Voxel Settings

Resolution:

☐ Minecraft-style color continuity

Custom Tool, Python



- Custom Tool, Python
- PLY to Voxel USD
- Multiple voxel size variations
- Vertex Color Usage
- Configurable shapes (cube/sphere)

04

REAL-TIME DEPLOYMENT

